



VAS VÁRMEGYEI SZAKKÉPZÉSI CENTRUM  
HORVÁTH BOLDIZSÁR  
KÖZGAZDASÁGI ÉS INFORMATIKAI  
TECHNIKUM

A 5 0613 12 03 számú Szoftverfejlesztő és –tesztelő vizsgaremek

## **A szoftveralkalmazás dokumentációja**

Készítették:

FUISZ MÁTÉ  
MÉSZÁROS GERGŐ

**SZOMBATHELY**

**2025**

## Tartalomjegyzék:

|                                                  |    |
|--------------------------------------------------|----|
| Szoftver célja .....                             | 5  |
| A program célja: .....                           | 5  |
| Felhasználási terület – célközönség:.....        | 5  |
| Célközönség: .....                               | 5  |
| • Vendégek:.....                                 | 5  |
| • Szépségszalon dolgozói:.....                   | 5  |
| Munkamegosztás: .....                            | 6  |
| Komponenseinek technikai leírása .....           | 7  |
| Szépségéden logó: .....                          | 7  |
| Frontend:.....                                   | 7  |
| Asztali alkalmazás: .....                        | 22 |
| Adatbázis: .....                                 | 25 |
| Program tesztek: .....                           | 27 |
| Backend tesztek: .....                           | 27 |
| Asztali alkalmazásban lévő adatvizsgálat: .....  | 28 |
| Működésének műszaki feltételei .....             | 29 |
| A program elindításának folyamata: .....         | 29 |
| Használatának rövid bemutatása .....             | 30 |
| Webes rész:.....                                 | 30 |
| Asztali alkalmazás: .....                        | 31 |
| Bejelentkezés a felhasználói fiókba:.....        | 32 |
| Bejelentkezés az admin/szakember fiókjába: ..... | 32 |
| Továbbfejlesztési lehetőségek .....              | 33 |
| Sötét mód:.....                                  | 33 |
| Nyelvek: .....                                   | 33 |
| Webshop: .....                                   | 33 |

|                       |    |
|-----------------------|----|
| Értékelések: .....    | 33 |
| Adat módosítás: ..... | 33 |
| Források: .....       | 34 |

## Bevezetés

A projektünk fő inspirációja a szentgotthárdi Centrum Szalon volt, ami egy olyan szolgáltatásközpont, ahol olyan vállalkozásokat érnek el a betérő vendégek, amelyek szépségipari és egészségügyi szolgáltatásokat biztosítanak, mint például fogászat, fodrászat, szemhélytetroválás. Mivel különböző üzemeltetők által működtetett vállalkozások működnek az épületben az elérhetőségek és ezen belül is az időpontfoglalás nincsen egységesen kezelve, tehát külön felületen és/vagy külön módokon történik. Mi szerettünk volna egy olyan szépségszalont ötletével elődrukkolni, amelynél hasonló típusú szolgáltatások találhatók, de egyetlen felületen belül történik az időpontfoglalás. Így jött nekünk az ötlet, hogy készítsünk egy szépségszalont, amihez tartozik egy online felület, ahol az összes szolgáltatás között lehet böngészni, valamint időpontokat foglalni ezekre. Valamint elérhető az oldalon egy galéria, ahol az elkészített munkákat tudják megtekinteni, valamint bizalmat gyűjteni az érdeklődők.

## **Szoftver célja**

### **A program célja:**

A weboldal célja, hogy egy modern, átlátható és felhasználóbarát online felületet biztosítson a Szépség Éden vendégei számára, ahol kényelmesen böngészhetnek a szolgáltatások között, megtekinthetik az elérhető időpontokat és szakembereket, és egyszerűen lefoglalhatják a számukra legideálisabb időpontot, kezelést. A rendszer célja, hogy megkönnyítse a zsúfolt hétköznapiakban az időpontfoglalást, csökkentse a várakozási időket, és javítsa felhasználói/ügyfélélményt mind az új, mind a visszatérő vendégek számára.

A program nem csak a vendégek, hanem a szalonban dolgozó szakemberek számára is kedvező, a számukra kialakított asztali alkalmazás segítségével könnyen nyomon követhetik a napi-heti beosztásukat, lemondhatják az időpontokat. optimalizálhatják munkájukat, úgy, hogy kevesebb időt kell adminisztrációra fordítaniuk. Ezáltal hatékonyabbá válik a munkaszervezés, kiszámíthatóbb a napirend, és több figyelem juthat magára az ügyfélre.

### **Felhasználási terület – célközönség:**

Az oldal egy szépségszalon számára készült, ahol különböző szolgáltatások például fodrászat, manikűr, pedikűr, kozmetika, gyantázás és számos több szolgáltatásra időpontra történő foglalás szükséges.

A felület elérhető mobiltelefonon, táblagépen és egyaránt asztali számítógépen is, így a vendégek bárhol könnyen elérhetik a honlapot és könnyen, pár kattintással tudnak időpontot foglalni. Az ott dolgozók a saját időpontjaikat egy számítógépes programban tudják kezelni.

### **Célközönség:**

- **Vendégek:**

- Olyan emberek, akik kényelmesen és online szeretnék időpontot foglalni az otthonuk kényelmében.
- Elfoglalt hétköznapi felhasználók, akik gyors és egyszerű foglalási megoldást keresnek.
- Digitálisan nyitott, akár a fiatalabb generáció, akik már hozzászoktak az online szolgáltatásokhoz.

- **Szépségszalon dolgozói:**

- Szakemberek, akik szeretnék átláthatóbbá tenni a beosztásukat.
- Olyan szolgáltatók, akik professzionálisabb ügyfélélményt szeretnék biztosítani a digitális jelenléttel.

## Munkamegosztás:

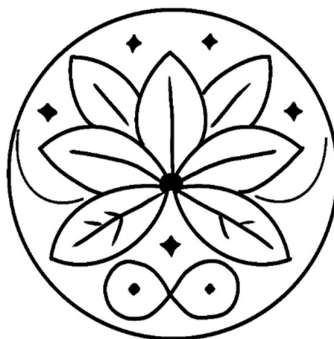
|                                    |                            |
|------------------------------------|----------------------------|
| Adatbázis fejlesztése, kezelése    | Fuisz Máté, Mészáros Gergő |
| Backend fejlesztés                 | Fuisz Máté                 |
| Frontend fejlesztés                | Mészáros Gergő             |
| Dizájn megtervezése, megvalósítása | Mészáros Gergő             |
| Asztali alkalmazás fejlesztés      | Fuisz Máté                 |
| Program tesztelés                  | Fuisz Máté, Mészáros Gergő |
| Dokumentáció elkészítése           | Fuisz Máté, Mészáros Gergő |

A kommunikációt Discordon oldottuk meg napi szintű egyeztetésekkel. A kódot GitHubon kezeltük, hogy a csapatunk mindkét tagja könnyen hozzáférhessen a változtatásokhoz.

## Komponenseinek technikai leírása

### Szépségéden logó:

A szépségéden logója, saját kezűleg lett tervezve, rajzolva. Az elkészítéséhez a „Procreate” nevű applikációt használtuk.



*Szépség Éden*

### Frontend:

#### Menüsáv:

A menüsávnak két állapota van, bejelentkezés előtti, utáni. Amikor még nincs „user”, addig a menüsávban nem jelenik meg a „Foglalás”, illetve a „Profil” menüpont. Ez a következőképpen lett kezelve:

```
{user && <Link to="/foglalas">FOGLALÁS</Link>}  
{user && <Link to="/profil">PROFIL</Link>}  
{!user && <Link to="/regisztracio">REGISZTRÁCIÓ</Link>}
```

*Dinamikus menü*

Ebben a kódrészletben látszik, hogy a kód megvizsgálja azt, hogy létezik-e felhasználó.

A privát útvonalak védettek, tehát ha profil nélkül kíván a vendég időpontot foglalni <http://localhost:5173/foglalas> ezt a címet a böngészőbe gépelve, akkor a regisztrációs oldalra fogja dobni.

```
<Route path="/foglalas" element={  
  user ? <Foglalas /> : <Navigate to="/regisztracio" replace />  
} />  
<Route path="/foglalas/:id" element={  
  user ? <BookingPage /> : <Navigate to="/regisztracio" replace />  
} />  
<Route path="/profil" element={  
  user ? <Profil /> : <Navigate to="/regisztracio" replace />  
} />  
<Route path="/regisztracio" element={<RegistrationLogin />} />
```

*Privát útvonalak védelme*

A menüsáv telefonos nézetben, egy lenyíló menüvé alakul át, amit JavaScript segítségével oldottunk meg:

```

export function openmenu() {
  let menuItems = document.querySelectorAll('.menu-list')
  let menuToggler = document.querySelector('.logo')

  menuToggler.onclick = function () {
    menuToggler.classList.toggle('open')
    let s = 0
    for (const menuItem of menuItems) {
      setTimeout(function () {
        menuItem.classList.toggle('open')
      }, s)
      s += 50
    }
  }
}

```

*Lenyíló menü telefonos nézetben*

## **Regisztráció / Bejelentkezés:**

### **Felhasználó elmentése:**

A felhasználói profil elmentéshez két megoldást használtunk az egyik a UserContext használatával működik, a másik pedig a „sessionStorage” használatával. Bejelentkezés esetén a weboldal megjegyzi a felhasználó adatait, ami szükséges az időpontok foglalásához, kezeléséhez, valamint az oldal megfelelő működéséhez.

### **Regisztráció:**

Amikor a vendég regisztrálni kíván akkor a biztonság érdekében a jelszónak meg kell felelnie a következő kritériumoknak:

- Minimum 8 karakter hosszúnak kell lennie,
- Tartalmaznia kell legalább egy nagybetűt,
- Tartalmaznia kell legalább egy számjegyet.

A telefonszámnak legalább 10 karakter hosszúnak kell lennie és csak számjegyeket tartalmazhat.

Helyes példa: 0123456789

Rossz példa: +3612345678

Az Email cím csak helyes formátumban elfogadott, valamint a vezetéknév minimum karakterszáma 2, a keresztnév pedig 3.

```

if (!emailRegex.test(email)) {
    alert("Az email cím formátuma érvénytelen!");
    return;
}

if (vnev.length < 2 || /^[^a-zA-ZáéíóöőúüűÁÉÍÓÖŰÚŰ]/.test(vnev)) {
    alert("A vezetéknévnek legalább 2 karakter hosszúnak kell lennie és csak betűket tartalmazhat.");
    return;
}

if (knev.length < 3 || /^[^a-zA-ZáéíóöőúüűÁÉÍÓÖŰÚŰ]/.test(knev)) {
    alert("A keresztnévnek legalább 3 karakter hosszúnak kell lennie és csak betűket tartalmazhat.");
    return;
}

if (!phoneRegex.test(phone)) {
    alert("A telefonszám csak számokat tartalmazhat, legalább 10 karakter hosszúnak kell lennie");
    return;
}

```

*Validáció*

```

if (password.length < 8) {
    alert("A jelszónak minimum 8 karakter hosszúnak kell lennie!");
    return;
}

if (!/[A-Z]/.test(password)) {
    alert("A jelszónak tartalmaznia kell legalább egy nagybetűt!");
    return;
}

if (!/\d/.test(password)) {
    alert("A jelszónak tartalmaznia kell legalább egy számjegyet!");
    return;
}

if (!passwordRegex.test(password)) {
    alert("Ilyen karaktert nem tartalmazhat!");
    return;
}

if (password !== confirmPassword) {
    alert("A két jelszó nem egyezik");
    return;
}

```

*Jelszó ellenőrzése*

### **Dinamikus oldalváltás: Regisztráció és Bejelentkezés:**

Ez a komponens nem külön oldalként kezeli a regisztrációt és a bejelentkezést, hanem egy „useState” alapú dinamikus megoldást használ, kattintásra váltogatja a megjelenített formot.

```
const [isLogin, setIsLogin] = useState(false);

const toggleForm = () => {
  setIsLogin((prev) => !prev);
};

return (
  <div>
    <Menu />
    {isLogin ? (
      <Login toggleForm={toggleForm} />
    ) : (
      <Regisztracio toggleForm={toggleForm} />
    )}
    <Footer />
  </div>
);
```

*Dinamikus oldalváltás*

Mindkét komponens használ egy linket, ami a „toggleForm” függvényt hívja meg. Ezáltal tud a felhasználó a formok között váltogatni.

```
<p className="toggle-link" onClick={toggleForm}>
  Nincs még profilom, Regisztrálok.
</p>
```

*Bejelentkezési felületen megjelenő link*

```
<p className="toggle-link" onClick={toggleForm}>
  Már van profilom, Bejelentkezek.
</p>
```

*Regisztrációs felületen megjelenő link*

## Profil:

### A felhasználó adatainak megjelenítése:

Az adatokat (Név, Email) a „sessionStorage”-ból tölti be automatikusan, ha nincs adat akkor a betöltési állapot jelenik meg.

```
useEffect(() => {
  const storedUser = sessionStorage.getItem("user");
  if (storedUser) {
    setUser(JSON.parse(storedUser));
  }
}, []);
```

*Felhasználó adatainak megjegyzése*

```

{user ? (
  <>
  <p><strong>Név:</strong> {user.Firstname} {user.Surname}</p>
  <p><strong>Email:</strong> {user.Email}</p>
</>
) : (
  <p>Felhasználói adatok betöltése...</p>
)}

```

*Felhasználó adatainak megjelenítése*

### A felhasználó foglalásainak megjelenítése:

A vendég a saját foglalásait egy naptár segítségével tudja megtekinteni, a hétvégék le vannak tiltva a „tileDisabled” segítségével.

```

tileDisabled=(({ date }) => {
  const nap = date.getDay();
  return nap === 0 || nap === 6;
})

```

*A naptárban letiltott dátumok (hétvégék)*

A kiválasztott dátum alapján API hívás történik, ami visszaadja az adott napra eső foglalásokat. A foglalások kártyában jelennek meg a vendég számára fontos információk. Abban az esetben, ha nincs foglalás az adott napon, akkor egy „Nincs foglalás erre a napra.” felirat jelenik meg a kártyák helyett.

| 2025. április |    |     |    |    |     |    |
|---------------|----|-----|----|----|-----|----|
| H             | K  | SZE | CS | P  | SZO | V  |
| 31            | 1  | 2   | 3  | 4  | 5   | 6  |
| 7             | 8  | 9   | 10 | 11 | 12  | 13 |
| 14            | 15 | 16  | 17 | 18 | 19  | 20 |
| 21            | 22 | 23  | 24 | 25 | 26  | 27 |
| 28            | 29 | 30  | 1  | 2  | 3   | 4  |

Kiválasztott dátum: 4/23/2025

*Naptár grafikus ábrázolása*

```

useEffect(() => {
  if (oldDate !== date) {
    //console.log(date)
    //console.log(oldDate)
    oldDate = date

    const formattedDate = `${date.getFullYear()}-${String(date.getMonth() + 1).padStart(2, '0')}-${String(date.getDate()).padStart(2, '0')}`;

    fetch(`https://localhost:44385/api/Reservation/vendeg/${UserData.Id}?Date=${formattedDate}`)
      .then(response => {
        if (!response.ok) {
          throw new Error("Hiba történt az időpontok lekérdezésekor!")
        }
        return response.json()
      })
      .then(data => {
        setDateInfo(data)
      })
      .catch((error) => console.log(error))
  }
},)

```

*Időpontokat lekérdező API hívás*

```

<p>Kiválasztott dátum: {date.toLocaleDateString()}</p>
<div className="foglalt-idopontok-container">
  {dateInfo.length > 0 ? (
    dateInfo.map((item, index) => (
      <div key={index} className={`foglalas-kartya ${selectedBookingId === item.Id ? "selected" : ""}`}
        onClick={() => setSelectedBookingId(item.Id)}>
        <p><strong>Időpont:</strong> {item.Idopont}</p>
        <p><strong>Szolgáltatás:</strong> {item.Szolgalattas}</p>
        <p><strong>Szakember:</strong> {item.Szakember}</p>
        <p><strong>Ár:</strong> {item.Ar} Ft</p>
        <p><strong>Hossz:</strong> {item.Hossz} perc</p>
      </div>
    ))
  ) : (
    <p>Nincs foglalás erre a napra.</p>
  )}

```

*Felhasználó foglalásainak megjelenítése*

|                                                                                                                                          |                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Időpont:</b> 11:00<br><b>Szolgáltatás:</b> Hajvágás<br><b>Szakember:</b> Fodrász Fanni<br><b>Ár:</b> 5000 Ft<br><b>Hossz:</b> 45 perc | <b>Időpont:</b> 13:00<br><b>Szolgáltatás:</b> Manikűr<br><b>Szakember:</b> Lakkozó Lara<br><b>Ár:</b> 4000 Ft<br><b>Hossz:</b> 30 perc |
|------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|

*Vendég számára megjelenő információk a foglalásáról*

## Foglalás lemondás:

A kattintással kiválasztott időpontot egy gombnyomással le lehet mondani, a törlés után a foglalások listája automatikusan frissül, így elkerülve a már törölt időpontok megjelenítését. A véletlenszerű időpont törlés elkerülése miatt, szükséges megerősíteni a törlést.

```
<button
  className="cancel-btn"
  disabled={!selectedBookingId}
  onClick={() => {
    if (!selectedBookingId) return;

    const confirmed = window.confirm("Biztosan törölni szeretnéd a foglalást?");
    if (!confirmed) return;

    fetch("https://localhost:44385/api/Reservation/${selectedBookingId}", {
      method: "DELETE"
    })
      .then((res) => {
        if (!res.ok) throw new Error("Törlés sikertelen!");
        alert("Foglalás sikeresen törölve!");

        const formattedDate = `${date.getFullYear()}-${String(date.getMonth() + 1).padStart(2, '0')}-${String(date.getDate()).padStart(2, '0')}`;

        return new Promise((resolve) => setTimeout(resolve, 300)).then(() =>
          fetch("https://localhost:44385/api/Reservation/vendeg/${user.Id}?Date=${formattedDate}")
        ); //elkerüljük a 404-es hibát a delay segítségével
      })
      .then(res => res.json())
      .then(data => {
        setDateInfo(data);
        setSelectedBookingId(null); // reset
      })
      .catch((err) => {
        console.error("Törlés hiba:", err);
        alert("Hiba történt a törlés közben!");
      });
  })
>
  Foglalás lemondása
</button>
```

*Időpont törlésének kódja*

## Jelszó módosítása:

Három mező használatával történik a változtatás. A jelszavak nem jelennek meg a képernyőn a biztonságos inputmezők miatt.

### Jelszó változtatás

Itt módosíthatod a jelszavad.

Jelszó módosítása

*Jelszováltoztatási felület*

A „Jelszó módosítása” gomb megnyomásakor meghívódik a „jelszoChange()” függvény, amiben kezelve van, hogy a jelszó miket és miket nem tartalmazhat, valamint az API hívás ami lecseréli a régi jelszót a kívánt jelszóra.

```

if (ujJelszo !== ujJelszoMeg) {
  alert("A két új jelszó nem egyezik!");
  return;
}

if (ujJelszo.length < 8) {
  alert("A jelszónak minimum 8 karakter hosszúnak kell lennie!");
  return;
}

if (!/[A-Z]/.test(ujJelszo)) {
  alert("A jelszónak tartalmaznia kell legalább egy nagybetűt!");
  return;
}

if (!/\d/.test(ujJelszo)) {
  alert("A jelszónak tartalmaznia kell legalább egy számjegyet!");
  return;
}

const passwordRegex = /^[A-Za-z0-9.*!$,%&+@#<>)\_-=\/\:\]{8,}$/;
if (!passwordRegex.test(ujJelszo)) {
  alert("A jelszó csak megengedett karaktereket tartalmazhat.");
  return;
}

```

*Új jelszó ellenőrzése*

```

const user = JSON.parse(sessionStorage.getItem("user"));

fetch(`https://localhost:44385/api/User/${user.Id}`, {
  method: "PATCH",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    OldPasswd: regiJelszo,
    NewPasswd: ujJelszo
  })
})
.then((response) => {
  if (!response.ok) {
    if (response.status === 401) {
      alert("Hibás a jelenlegi jelszó!");
    } else {
      throw new Error(`HTTP hiba! Státuszkód: ${response.status}`);
    }
  }
  return;
})

alert("Sikeres jelszómódosítás!");
return response.json();
})
.then((data) => {
  if (!data) return;
  console.log(data);
  localStorage.setItem("user", JSON.stringify(data));

  document.getElementById("regijelszo").value = "";
  document.getElementById("ujjelszo").value = "";
  document.getElementById("ujjelszomegegyszer").value = "";
})
.catch((error) => {
  console.error("Hiba történt: ", error);
  alert("Szerver hiba. Próbáld meg később!");
});

```

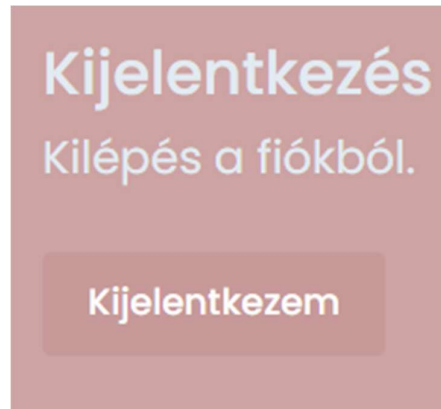
*Jelszómódosítást végző API hívás*

### Kijelentkezés:

Egy egyszerű gombnyomással az adatok törlődnek a „sessionStorage”-ból és visszadob az alkalmazás kezdőoldalára.

```
function logout() {  
  sessionStorage.clear();  
  window.location.href = "/"  
}
```

*Adatok törlése és a főoldal megjelenítése*



*Kijelentkezés gomb*

### Foglalási oldal (BookingPage):

Ez a komponens a foglalási folyamat központi része. Ez az oldal felelős a szolgáltatás kiválasztása után történő szakember, dátum és időpont kiválasztásáért.

### Szolgáltatás betöltése és azonosítása:

A „useParams()” segítségével kiolvassuk az adott szolgáltatás azonosítóját, majd lekérjük a hozzá tartozó adatokat.

```
const services = [  
  { id: 1, name: "Hajfestés", image: hajfestesImg },  
  { id: 2, name: "Hajvágás", image: hajvagasiImg },  
  { id: 3, name: "Manikűr", image: manikurImg },  
  { id: 4, name: "Körömfestés", image: koromfestesImg },  
  { id: 5, name: "Pedikűr", image: pedikurImg },  
  { id: 6, name: "Masszázs", image: masszazsiImg },  
  { id: 7, name: "Arckezelés", image: arckezelesImg },  
  { id: 8, name: "Gyantázás", image: gyantazasiImg },  
];
```

*Szolgáltatások listája*

```
const { id } = useParams();
```

*"id" konstans deklarációja*

```
//szolgaltatas lekerese
useEffect(() => {
  fetch("https://localhost:44385/api/Service")
    .then((res) => res.json())
    .then((data) => {
      setServices(data);
      const found = data.find((s) => s.Id === parseInt(id));
      if (found) setService(found);
    })
    .catch((err) => console.error("Szolgáltatás hiba:", err));
}, [id]);
```

*A szolgáltatások lekérését végző API hívás*

### Naptár megjelenítése es dátumválasztás:

A komponens tartalmaz egy „react-calendar” alapú naptárat, a hétvégi időpontok le vannak tiltva, mivel a hétvégék szabadnapok. A kiválasztott dátum hatással van az elérhető szakemberekre, valamint időpontokra.

```
<Calendar
  onChange={setDate}
  value={date}
  minDate={new Date()}
  tileDisabled={({ date }) => {
    const nap = date.getDay();
    const today = new Date();
    today.setHours(0, 0, 0, 0);
    const isWeekend = nap === 0 || nap === 6;
    const isToday = date.getTime() === today.getTime();
    return isWeekend || isToday;
  }}
/>
```

*"react-calendar" kódja*

### Szakemberek listázása a kiválasztott szolgáltatás alapján:

Miután a szolgáltatás betöltődött, a rendszer lekéri azokat a szakembereket, akik el tudják végezni az adott szolgáltatást.

```
//szakemberek betoltese
useEffect(() => {
  if (!service) return;
  fetch(`https://localhost:44385/api/Userservices?id=${service.Id}`)
    .then((res) => res.json())
    .then((data) => setSzakemberek(data))
    .catch((err) => console.error("Szakember hiba:", err));
}, [service]);
```

*A szakemberek betöltéséért felelős API hívás*

A felhasználó vizuálisan választhat az elérhető szakemberek közül.

Válassz szakembert:

Borbély Bence

Fodrász Fanni

Haj Hanna

*Szakemberek megjelenítése*

### Foglalások és munkaidő lekérdezése:

A kiválasztott szakember és dátum alapján két további adatforrás kerül lekérdezésre. Az adott napra eső már meglévő foglalások, valamint a szakembernek az adott napra vonatkozó munkaidő-beosztása.

```
//foglalások betoltese
useEffect(() => {
  if (!selectedUserId || !date) return;

  const formattedDate = `${date.getFullYear()}-${String(date.getMonth() + 1).padStart(2, '0')}-${String(date.getDate()).padStart(2, '0')}`;

  fetch(`https://localhost:44385/api/Reservation/dolgozo/${selectedUserId}?Date=${formattedDate}`)
    .then((res) => res.json())
    .then((data) => {
      setReservations(data);
    })
    .catch((err) => console.error("Foglalások betöltése sikertelen:", err));
}, [selectedUserId, date]);
```

*Már meglévő foglalások letöltése*

```
//munkaido betoltese
useEffect(() => {
  if (!selectedUserId || !date) return;

  fetch(`https://localhost:44385/api/Workinghours/${selectedUserId}`)
    .then((res) => res.json())
    .then((data) => setWorkingHours(data))
    .catch((err) => console.error("Munkaidő hiba:", err));
}, [selectedUserId, date]);
```

*Szakemberek munkaidejének betöltése*

### Szabad időpontok kiszámítása:

Ez a funkció a komponens egyik legösszetettebb és legfontosabb része. A rendszer feladata az, hogy a szakember munkaidejéből, valamint az adott napra vonatkozó már meglévő foglalásokból kiindulva meghatározza azokat az időpontokat, amelyek még szabadok, és amelyekbe maradéktalanul belefér a kiválasztott szolgáltatás.

A számítás során a következő szempontokat veszi figyelembe:

- A szolgáltatás csak olyan időpontra foglalható, amely a szakember munkaidején belül kezdődik és be is fejeződik.
- Az algoritmus dinamikusan kezeli azt az esetet is, amikor a szakember többféle időtartamú szolgáltatást végez egy napon belül. A meglévő foglalásoknál minden egyes szolgáltatás időtartamát külön-külön figyelembe veszi, így pontosan kiszámítja, mely időpontok ütköznek és melyek férnek még bele a szabad kapacitásba.
- A rendszer kizárja azokat az időpontokat is, amelyek közvetlenül a meglévő foglalás előtt vannak, de ahová a kiválasztott szolgáltatás időtartama már nem férne be ütközés nélkül.
- Az időpontok kiszámítása 30 perces bontásban történik, a szolgáltatás időtartama alapján történő összevetéssel.

```
//szabad idopontok kiszamitasa
useEffect(() => {
  if (!selectedUserId || !date || !service || workingHours.length === 0) return;

  const day = date.getDay() === 0 ? 7 : date.getDay();
  const todayHours = workingHours.find((d) => d.Daynumber === day);
  if (!todayHours) {
    setAvailableTimeSlots([]);
    return;
  }

  const [startH, startM] = todayHours.Openingtime.split(":").map(Number);
  const [endH, endM] = todayHours.Closingtime.split(":").map(Number);
  const startMin = startH * 60 + startM;
  const endMin = endH * 60 + endM;

  const slotDuration = 30;
  const serviceDuration = service.Duration;

  const slots = [];
  for (let min = startMin; min + serviceDuration <= endMin; min += slotDuration) {
    const h = String(Math.floor(min / 60)).padStart(2, "0");
    const m = String(min % 60).padStart(2, "0");
    slots.push(`${h}:${m}`);
  }

  const blocked = new Set();

  reservations.forEach((item) => {
    const duration = item.SzolgalatasiHossz || 30;
    const [h, m] = item.FoglaltIdopont.split(":").map(Number);
    const start = h * 60 + m;
    const slotCount = Math.ceil(duration / slotDuration);

    // blokkoljuk a foglalas idejet es utana levo idopontokat
    for (let i = 0; i < slotCount; i++) {
      const total = start + i * slotDuration;
      const hh = String(Math.floor(total / 60)).padStart(2, "0");
      const mm = String(total % 60).padStart(2, "0");
      blocked.add(`${hh}:${mm}`);
    }

    // blokkoljuk az elotte levo idopontokat, amik nem fennek bele az idobe
    const reverseBlockCount = Math.ceil(serviceDuration / slotDuration) - 1;
    for (let j = 1; j <= reverseBlockCount; j++) {
      const backMin = start - j * slotDuration;
      if (backMin >= startMin) {
        const bh = String(Math.floor(backMin / 60)).padStart(2, "0");
        const bm = String(backMin % 60).padStart(2, "0");
        blocked.add(`${bh}:${bm}`);
      }
    }
  });

  const free = slots.filter((slot) => !blocked.has(slot));
  setAvailableTimeSlots(free);
}, [selectedUserId, date, reservations, service, workingHours]);
```

*A nem elérhető időpontok kiszámítása*

Ez az algoritmus biztosítja, hogy a felhasználó kizárólag valóban foglalható, konfliktusmentes időpontokból választhasson.

### Foglalás elküldése:

A felhasználó a naptárból választott dátum, a kiválasztott szakember és egy szabad időpont kiválasztása után tudja elküldeni a foglalást. A POST kérés tartalmazza az összes szükséges adatot (felhasználó ID, szakember ID, szolgáltatás ID, dátum, időpont), és sikeres mentés után visszairányítja a felhasználót a foglalásait tartalmazó oldalra.

```
const handleSubmit = (e) => {
  e.preventDefault();
  if (!selectedUserId) {
    alert("Kérlek válassz szakembert!");
    return;
  }
  if (!time) {
    alert("Kérlek válassz időpontot!");
    return;
  }
  const user = JSON.parse(sessionStorage.getItem("user"));
  const payload = {
    Worker_Id: selectedUserId,
    Guest_Id: user.Id,
    Service_Id: service.Id,
    Datetime: date.toLocaleDateString("sv-SE"),
    Appointment: time
  };

  fetch("https://localhost:44385/api/Reservation", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(payload)
  })
  .then((res) => {
    //console.log(payload);
    if (!res.ok) throw new Error("Foglalás sikertelen!");
    return res.json();
  })
  .then(() => {
    alert(
      `Foglalás sikeres:\nSzolgáltatás: ${service.Name}\nDátum: ${date.
        toLocaleDateString()}\nIdőpont: ${time}\nÁr: ${service.Price}Ft`
    );
    window.location.href = "/foglalas";
  })
  .catch((err) => {
    console.error("POST hiba:", err);
    alert("Hiba történt a foglalás mentésekor.");
  });
};

if (!service) return <div>Szolgáltatás betöltése...</div>;
```

*A foglalás elküldése*

## Mobilbarát dizájn, reszponzivitás:

Az alkalmazás teljes mértékben reszponzív, úgy lett kialakítva, hogy minden eszközön (telefon, asztali gép, táblagép) kényelmesen használható legyen. A dizájn automatikusan alkalmazkodik különböző képernyőméretekhez. A megvalósításhoz „CSS media query”-ket használtunk.

Példa:



*Tájékoztató számítógépes nézetben*



*Tájékoztató telefonos nézetben*

## Regisztráció

☐ Elfogadom az általános szerződési feltételeket!

[Már van profilom, Bejelentkezek.](#)

Az időpont foglaláshoz szükséges felhasználói fiókkal rendelkezni!

*Regisztráció számítógépes nézetben*

## Regisztráció

☐ Elfogadom az általános szerződési feltételeket!

[Már van profilom, Bejelentkezek.](#)

Az időpont foglaláshoz szükséges felhasználói fiókkal rendelkezni!

*Regisztráció telefonos nézetben*

### Felhasználói fiókok és rugalmas foglalási lehetőségek:

A rendszerünk úgy lett kialakítva, hogy egy felhasználói fiókhoz, egy időponthoz több foglalás is tartozhat, így nem szükséges minden egyes családtagnak külön profilt létrehozni. Ez akkor hasznos, amikor, a szülő a gyermekének foglal időpontot, egy időpontra több szolgáltatást is igénybe kívánnak venni (pl. anya és gyerek külön-külön, de egy időben mennek külön kezelésre, a dolgozó saját magának és családtagjainak is szeretne időpontot foglalni).

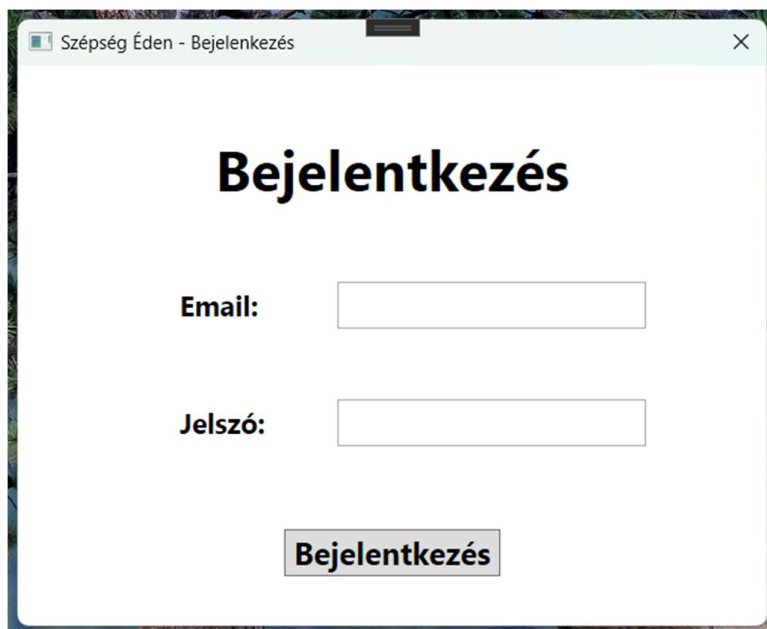
Példa:

- A vendég saját profilján keresztül lefoglalja 10:00-ra masszázst magának, 10:00-ra hajvágást gyermekének.
- Egy dolgozó foglal családtagjának egy időpontot, hogy neki ne kelljen külön profilt létrehozni.

### Asztali alkalmazás:

#### Bejelentkezés: (Bejelentkezés.xaml)

Az asztali alkalmazásba kizárólag az adminisztrátor és a dolgozók jogosultak belépni és módosításokat elvégezni.

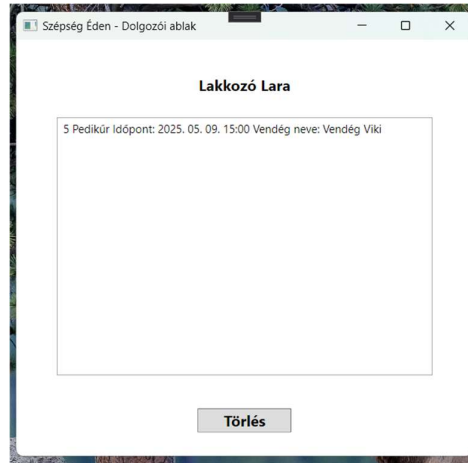


The image shows a screenshot of a desktop application window titled "Szépség Éden - Bejelentkezés". The window contains a login form with the heading "Bejelentkezés". Below the heading are two input fields: "Email:" and "Jelszó:". At the bottom of the form is a button labeled "Bejelentkezés".

*Bejelentkezés az asztali alkalmazásba*

### **Dolgozói felület: (Dolgozo.xaml)**

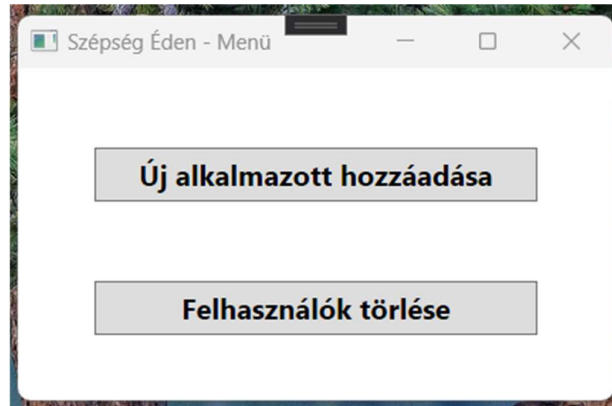
A felületen a dolgozók megtekinthetik a hozzájuk történt foglalásokat időrendi sorrendbe és ha szeretnék ki is törölhetik azokat.



*Dolgozó felülete, ahol megtekintheti, törölheti időpontjait*

### **Menü: (Menu.xaml)**

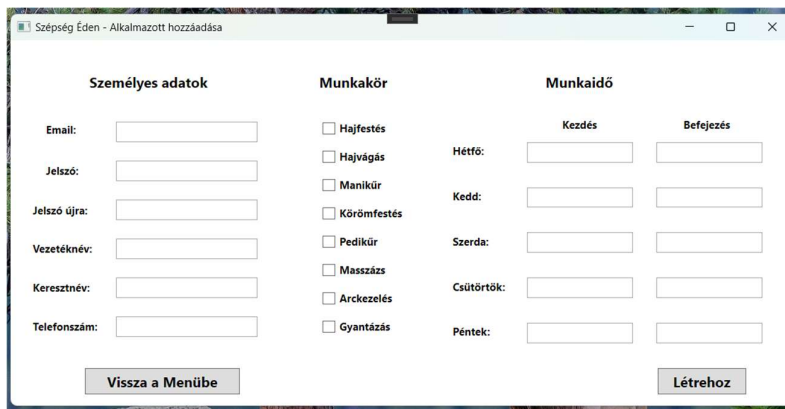
A menü oldalt csak az adminisztrátor érheti el és választhatja ki, hogy az alkalmazás melyik funkcióját kívánja használni.



*Admin kezelői felülete*

### Alkalmazottak hozzáadása: (Felvetel.xaml)

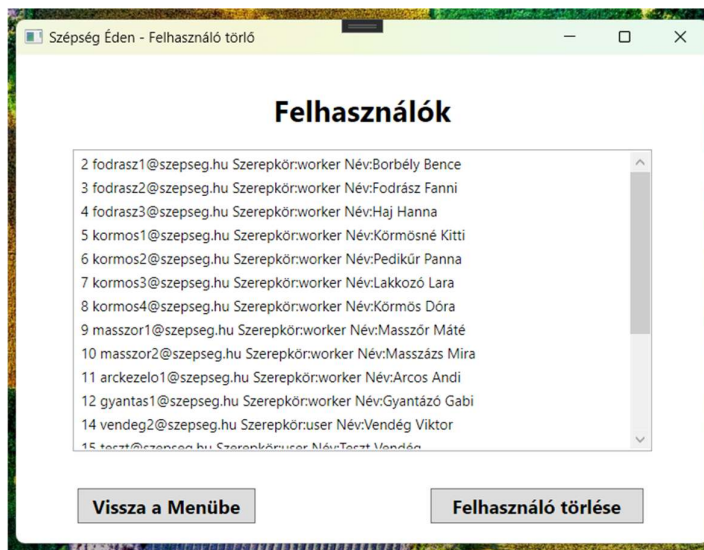
Az adminisztrátor új alkalmazottakat vehet fel a rendszerbe miután az összes mezőt helyesen kitöltötte.



*Ezen a fülön tud az admin új alkalmazottakat felvenni*

### Felhasználó törlő: (Torles.xaml)

Ezen a felületen kitörölhetjük a vendégek és a dolgozók profilját, az összes hozzátartozó adattal együtt.



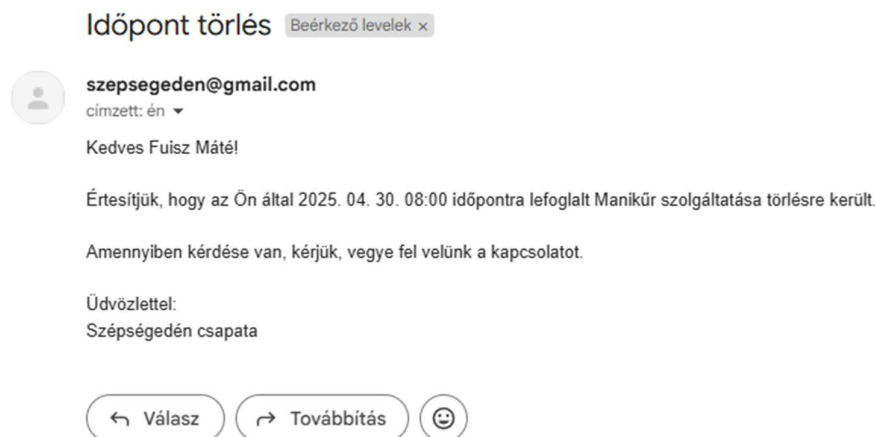
| ID | Email                 | Szerepkör | Név                |
|----|-----------------------|-----------|--------------------|
| 2  | fodrasz1@szepseg.hu   | worker    | Név:Borbély Bence  |
| 3  | fodrasz2@szepseg.hu   | worker    | Név:Fodrász Fanni  |
| 4  | fodrasz3@szepseg.hu   | worker    | Név:Haj Hanna      |
| 5  | kormos1@szepseg.hu    | worker    | Név:Körmösné Kitti |
| 6  | kormos2@szepseg.hu    | worker    | Név:Pedikűr Panna  |
| 7  | kormos3@szepseg.hu    | worker    | Név:Lakkozó Lara   |
| 8  | kormos4@szepseg.hu    | worker    | Név:Körmös Dóra    |
| 9  | masszor1@szepseg.hu   | worker    | Név:Masszór Máté   |
| 10 | masszor2@szepseg.hu   | worker    | Név:Masszázs Mira  |
| 11 | arckezelo1@szepseg.hu | worker    | Név:Arcos Andi     |
| 12 | gyantas1@szepseg.hu   | worker    | Név:Gyantázó Gabi  |
| 14 | vendeg2@szepseg.hu    | user      | Név:Vendég Viktor  |
| 15 | test@szepseg.hu       | user      | Név:Test Vendég    |

*Ezen a fülön tud az admin vendégeket, illetve dolgozókat törölni a rendszerből*

## Értesítő Email:

Esetek amikor a felhasználó értesítést kap Emailben:

- Amennyiben a dolgozó lemondja az időpontját.
- Amennyiben a vendég profilja törlésre kerül, és már rendelkezik jövőbeli időponttal.
- Amennyiben a szolgáltató profilja törlésre kerül (pl.: távozik a munkahelyről, elbocsátják).



*Email értesítés*

## Adatbázis:

A „szepsegeden” nevű adatbázisunk a következő táblákat tartalmazza:

- Days, tartalmazza a hét napjait.
- Reservation, eltárolja a foglalásokat.
- Role, a szerepek tárolásáért felelős (admin, dolgozó, vendég).
- Service, az elérhető szolgáltatásokat tárolja (pl.: manikűr, fodrász stb.)
- User, a felhasználók adatait tartalmazza.
- Userservices, A dolgozókhoz tartozó szolgáltatások kapcsolatát tartalmazza.
- Workinghours, A dolgozókhoz tartozó munkaidőket tárolja.

## Relációs modell:

Days (daynumber, name)

Reservation (id, worker\_id, guest\_id, service\_id, datetime, appointment)

Role (id, name)

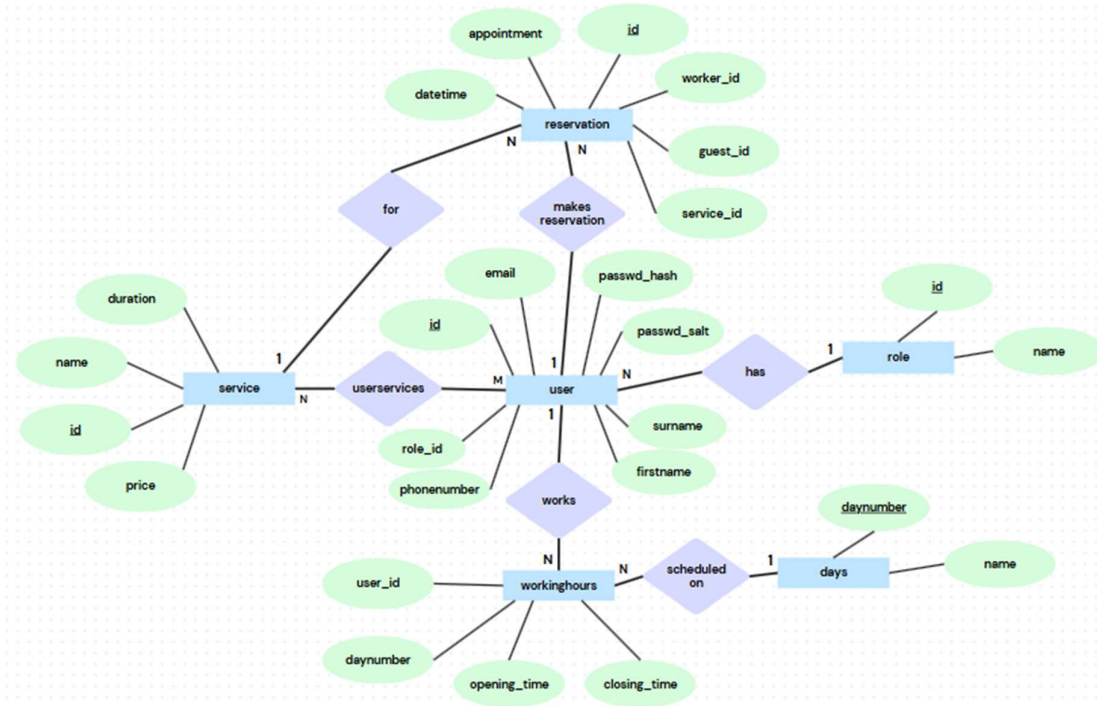
Service (id, name, price, duration)

User (id, email, passwd\_hash, passwd\_salt, surname, firstname, phonenumber, role\_id)

Userservices (user\_id, service\_id)

Workinghours (user\_id, daynumber, opening\_time, closing\_time)

## ER diagram:



## Program tesztek:

A backendünk tesztelésére a Postman nevű backend tesztelőt használtuk.

## Backend tesztek:

GET https://localhost:44385/ + ...

https://localhost:44385/api/Workinghours/2 Save

GET https://localhost:44385/api/Workinghours/2 Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
```

Test scripts are written in JavaScript, and are run after the response is received. Learn more about [tests scripts](#)

Snippets

Body Cookies Headers (10) Test Results (1/1) Status: 200 OK Time: 12 ms Size: 690 B Save Response

All Passed Skipped Failed

PASS Status code is 200

### Teszt #1

https://localhost:44385/api/Reservation/dolgozo/?Date=2025-04-14&id=1 Save

GET https://localhost:44385/api/Reservation/dolgozo/?Date=2025-04-14&id=1 Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

```
1 pm.test('Status code is 200', function () {
2   pm.response.to.have.status(200);
3 });
```

Test scripts are written in JavaScript, and are run after the response is received. Learn more about [tests scripts](#)

Snippets

Get a variable

Send a request

Status code: Code is 200

Body Cookies Headers (10) Test Results (1/1) Status: 200 OK Time: 10 ms Size: 397 B Save Response

All Passed Skipped Failed

PASS Status code is 200

### Teszt #2

https://localhost:44385/api/Service Save

GET https://localhost:44385/api/Service Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

```
1 pm.test('Status code is 200', function () {
2   pm.response.to.have.status(200);
3 });
```

Test scripts are written in JavaScript, and are run after the response is received. Learn more about [tests scripts](#)

Snippets

Get a variable

Send a request

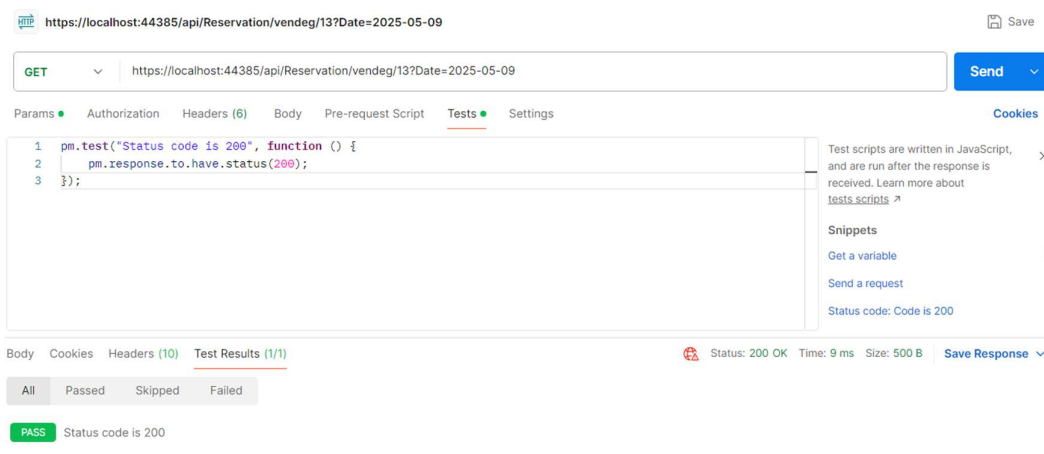
Status code: Code is 200

Body Cookies Headers (10) Test Results (1/1) Status: 200 OK Time: 7 ms Size: 829 B Save Response

All Passed Skipped Failed

PASS Status code is 200

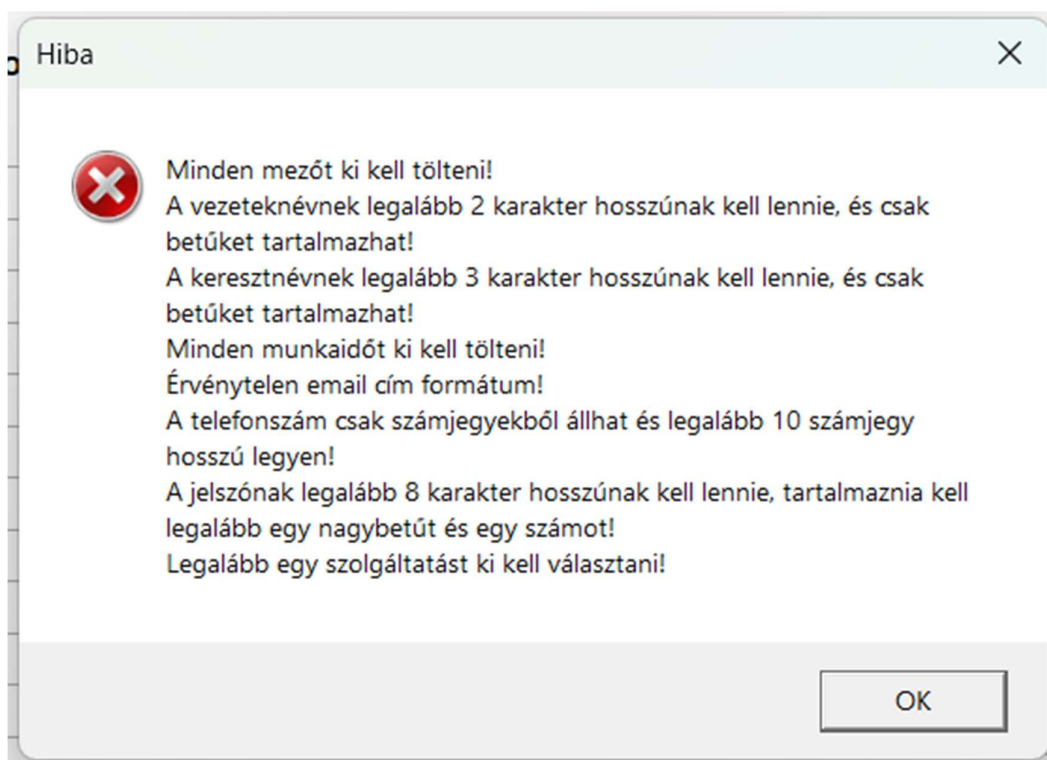
### Teszt #3



#### Teszt #4

### Asztali alkalmazásban lévő adatvizsgálat:

Abban az esetben, ha hiányos, rossz adatokat adunk meg új felhasználó felvételekor, akkor ez a hibaüzenet jelenik meg a képernyőn.



#### Hibaüzenet

## **Működésének műszaki feltételei**

A weboldal működésének több kritériuma is van. Elsősorban a működéshez szükséges a Node.js telepítése (ajánlott a legfrissebb verziót használni). A rendszer adatbázisa MySQL adatbázis. Kezeléséhez célszerű XAMPP szoftvercsaládot használni, ezen belül a phpMyAdmin felület szolgál az adatbázisok létrehozására, kezelésére és karbantartására. A backend oldal fejlesztése és futtatása, valamint az asztali alkalmazás a Microsoft Visual Studio 2019 segítségével történik. A frontend rész futtatásához szükség van a Vite build tool telepítésére, valamint a React könyvtárra.

### **A program elindításának folyamata:**

A XAMPP futtatása és a phpMyAdmin elindítása után következik az adatbázis importálása (szepsegeden.sql), ezután importálja az adatfeltöltő fájlt is, amit „init.sql” néven talál meg a „Adatbázis” mappában. Amikor már fut az adatbázis indítsa el a „Szepsegeden” mappában található Szepsegeden.sln fájlt, majd ellenőrizze le, hogy az App.config, Web.config fájlokban (Szepsegeden, SzepsegedenLibrary, SzepsegedenWPF) a „connectionStrings” portszáma megegyezik-e a XAMPP-ban a phpMyAdmin mellett felbukkanó portszámmal (leggyakoribb portszámok: 3306, 3307). Ha a két port megegyezik, akkor elindíthatja a programot. A Solution Explorerben először válassza ki a megfelelő startup projektet: backend indításához a Szepsegeden projektet, WPF alkalmazás indításához pedig a SzepsegedenWPF projektet kell beállítani. Ezután indításhoz a képernyő tetején található zöld háromszögre kell kattintani. Ügyeljen arra, hogy mindig a megfelelő projekt legyen kijelölve a Solution Explorerben, különben nem a kívánt alkalmazás fog elindulni. A frontend elindítása a következő módon zajlik: Nyissa meg azt a mappát, amiben az „src” nevű könyvtár is megtalálható, majd itt nyissa meg a parancssort és gépelje be az „npm run dev” parancsot. A parancs futtatása után megjelenik a konzolon egy link amire kattintva/vagy a böngészőbe másolva elindul a webes felület. Mindezek után a vendégek felhasználói felülete készen áll a használatra.

## **Használatának rövid bemutatása**

### **Webes rész:**

#### **Főoldal:**

A főoldalon a vendégeket egy modern, letisztult dizájn fogadja, egy sliderrel, amin figyelemfelkeltő képek szerepelnek a szolgáltatásokról. Az oldal közepén egy rövid ismertető található az online időpontfoglalásról, mellette egy “Foglalás” gombbal. Fontos: ha a felhasználó még nincs bejelentkezve, akkor ez a gomb automatikusan a regisztrációs oldalra irányítja, ahol létrehozhatja a fiókját. Bejelentkezett állapotban a gomb a foglalási oldalra navigálja a vendéget, ahol pár kattintással tud időpontot foglalni.

#### **Galéria:**

A galériában a vendégek, a szépségszalon szakemberei által elkészített munkákat tudják szemügyre venni.

#### **Regisztráció:**

A regisztrációs oldal két funkcióval szolgál. Az első, amikor az oldalra látogatónak még nincs felhasználói fiókja, akkor itt tud regisztrálni. A linkre kattintva előjön a másik funkció, ami az oldalra való bejelentkezés. A kért adatok helyes megadásával sikeresen tud regisztrálni és bejelentkezni. A bejelentkezés után új menüpontok jelennek meg a menüsávban (PROFIL, FOGLALÁS), a profil menüpont fog megjelenni.

#### **Profil:**

A Profil oldalon, több menü érhető el egymás alatt. A “Profil”-t kiválasztva a felhasználó meg tudja nézni a saját adatait.

A “Foglalásaim”-ra kattintva a vendég megtekintheti a foglalásait egy naptár segítségével, valamint a kívánsága szerint tudja azokat törölni is.

Abban az esetben, ha a felhasználó meg szeretné változtatni a jelszavát, könnyedén megteheti pár kattintással a “Jelszó változtatás” menüpontnál.

Kijelentkezni egyszerűen egy gomb megnyomásával lehet a “kijelentkezés” fülön.

#### **Foglalás:**

A “Foglalás” menüponton a vendégek böngészhetnek az elérhető szolgáltatások között, majd miután kiválasztották a számukra megfelelőt, kiválaszthatják a számukra ideális időpontot és szimpatikus szakembert, ezen az oldalon van megjelenítve a szolgáltatás időtartama valamint az ára is, sikeres foglalás után az időpontjuk megjelenik a „Profil” fülön.

**Asztali alkalmazás:****Szakember:**

Ha egy szakember jelentkezik be az alkalmazásba, akkor a saját időpontjait tudja megtekinteni és törölni, ebben az esetben a vendég emailt kap arról, hogy a foglalása törlésre került.

**Admin:****Új alkalmazott felvétele:**

Az admin új alkalmazottakat vehet fel ezen a fülön.

**Profilok törlése:**

Az admin itt törölheti a vendégeket, illetve a dolgozókat.

## **Bejelentkezés a felhasználói fiókba:**

### **A teszt felhasználó profil adatai:**

- **Email:** `teszt@szepseg.hu`
- **Jelszó:** `Teszt123`

A weboldal megnyitásakor a főoldal fogja köszönteni, a menüsávban több elérhető opció jelenik meg, válassza ki a “REGISZTRÁCIÓ” menüpontot. A regisztrációs oldalon menjen a „Már van profilom, bejelentkezek.” linkre, és gépelje be a megadott adatokat az email, jelszó mezőbe. Abban az esetben, ha új profilt szeretne regisztrálni, azt könnyen megteheti, ha visszamegy a regisztrációs oldalra.

## **Bejelentkezés az admin/szakember fiókjába:**

### **Admin adatai:**

- **Email:** `admin@admin.hu`
- **Jelszó:** `Admin123`

### **A teszt szakember profil adatai:**

- **Email:** `fodrasz2@szepseg.hu`
- **Jelszó:** `FodraszFanni123`

Az alkalmazás elindításakor, csak egyszerűen gépelje be a megfelelő adatokat és már használhatja is a programot.

## **Továbbfejlesztési lehetőségek**

### **Sötét mód:**

Mivel jelenleg az oldalon csak világos mód érhető el, ezért célszerű a későbbiekben a sötét mód fejlesztése.

### **Nyelvek:**

A weboldalon elérhető nyelvek bővítése, így később több vendég találhat az oldalra.

### **Webshop:**

Hasznos lehet egy webshop létrehozása, ahol a vendégek a szakemberek által használt termékeket tudják megvásárolni.

### **Értékelések:**

Egy olyan felület kialakítása, ahol a vendégek értékelni tudják a szakembereket, le tudják írni a véleményüket.

### **Adat módosítás:**

Egy olyan platform létrehozása, ahol a vendégek, illetve szakemberek, saját maguknak változtathatják meg a személyes adataikat, dolgozók esetében a munkaidejüket is.

## Források:

Galériához tartozó CSS:

<https://workik.com/best-gallery-designs-for-websites-with-html-css-and-javascript>

Ikonok:

<https://www.flaticon.com/>

Képek:

<https://unsplash.com/>

Térkép:

<https://www.embedmymap.com/>

Carousel:

<https://react-bootstrap.netlify.app/docs/components/carousel/>

React-slick-slider:

<https://react-slick.neostack.com/docs/example/swipe-to-slide>

Naptár:

<https://www.npmjs.com/package/react-calendar>

Háttér-minta:

<https://www.svgbackgrounds.com/set/free-svg-backgrounds-and-patterns/>